

//use//



React API

What is use()?

- A new native hook **introduced in React 19** that allows you to await promises directly in the render phase and **consume contexts declaratively**.
- Works seamlessly with **Suspense** for **loading states** and **Error Boundaries** for error handling.
- Helps you **write cleaner, declarative, and less verbose code** compared to `useState` + `useEffect`.

Basic Example -

Fetching Data Without useState and useEffect

Before (with useEffect and useState)

```
function UserProfile() {  
  const [user, setUser] = React.useState(null);  
  const [loading, setLoading] = React.useState(true);  
  const [error, setError] = React.useState(null);  
  
  React.useEffect(() => {  
    fetch('/api/user')  
      .then(res => res.json())  
      .then(data => setUser(data))  
      .catch(err => setError(err))  
      .finally(() => setLoading(false));  
  }, []);  
  
  if (loading) return <p>Loading user...</p>;  
  if (error) return <p>Error loading user: {error.message}</p>;  
  
  return <h1>Welcome, {user.name}!</h1>;  
}
```

Basic Example -

Fetching Data Without useState and useEffect

Now (with use() and Suspense)

```
import { use, Suspense } from 'react';

function fetchUser() {
  return fetch('/api/user').then(res => res.json());
}

function UserProfile() {
  const user = use(fetchUser());
  return <h1>Welcome, {user.name}!</h1>;
}

function App() {
  return (
    <Suspense fallback=<p>Loading user...</p>>
      <UserProfile />
    </Suspense>
  );
}
```


Advanced Example

Using **use()** Conditionally

```
function UserDetails({ showDetails }) {  
  if (showDetails) {  
    const user = use(fetch('/api/user').then(res => res.json()));  
    return <p>Email: {user.email}</p>;  
  }  
  return <p>User details are hidden.</p>;  
}
```

- `use()` can be called inside conditions or loops, unlike `useContext` or other hooks.

Advanced Example

Using use() with React Context

```
const ThemeContext = React.createContext('light');

function ThemedButton() {
  const theme = use(ThemeContext);
  return <button className={theme}>Click Me</button>;
}

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <ThemedButton />
    </ThemeContext.Provider>
  );
}
```

- You can consume Context values declaratively and even conditionally

Best Practices

 Do this	 Avoid this
Wrap components using <code>use()</code> in <code><Suspense></code>	Calling <code>use()</code> outside of render
Use Error Boundaries to catch promise rejections	Forgetting Suspense fallback
Stabilize Promises (avoid re-fetching)	Triggering a new Promise on each render
Use <code>use()</code> for read-only data fetching	Using <code>use()</code> for user-triggered events